Tabla de contenidos

1. Introducción
2. Procesos y núcleos
3. Paralelismo VS Concurrencia
4. *Threading*
    a. Usos de *threads*
    b. Creación de *threads*
        i. ¿Qué pasa con los `print`?
    c. Esperando otros *threads*: `join()`
    d. `is_alive()`
    e. *Daemons*
5. Timers

Introducción

Hasta el momento, los programas que hemos escrito solo ejecutan **una secuencia de instrucciones** a la vez. Siguen un único flujo que comienza, ejecuta instrucciones, y en algún momento termina.

Cada vez que queremos que el usuario ingrese un valor (por ejemplo, una opción en un menú), tenemos que ejecutar `input()` y esperar que el usuario responda. Mientras eso ocurre, nuestro programa debe pausarse mientras espera la respuesta. De la misma manera, si el usuario apreta una tecla y nuestro programa no está esperando que lo haga, no tiene cómo darse cuenta ni cómo recibir esa entrada. Esto es una consecuencia de que los programas que hemos escrito solo siguen un único flujo de instrucciones, y por lo tanto están atentos a sólo una cosa al mismo tiempo.

Muchos programas o aplicaciones que utilizamos realizan múltiples acciones simultáneamente. Cuando escribimos en un procesador de texto, al mismo tiempo se está ejecutando un código que revisa la ortografía. Cuando escribimos código fuente en una IDE, al mismo tiempo también se está ejecutando un código que revisa la sintaxis, que cambia los colores del texto y a veces nos hace sugerencias para completar lo que escribimos. Cuando estamos dentro de un juego, estamos usando teclas para mover a un personaje, pero también están pasando otros eventos en el ambiente del juego al mismo tiempo que movemos el personaje. Si tuviéramos que programar este comportamiento con las herramientas que tenemos, tendríamos que tener un ciclo que, por cada `input()` del usuario, ejecute métodos que revisen ortografía, sintaxis, que hagan sugerencias, que simulen el ambiente del juego, etc. Este comportamiento sería complicado de programar dentro de un programa que pone atención solo a un flujo de instrucciones.

La solución entonces, es que nuestro programa esté atento a múltiples instrucciones simultáneamente. Podemos conseguir esto teniendo "subprogramas": uno que ejecuta el método que revisa ortografía, uno que ejecuta el método que revisa sintaxis, uno que ejecuta el método que hace sugerencias, uno que simula el juego... pero todo dentro del mismo programa.

¿Cómo conseguir esto? El concepto de ***thread*** nos permite definir **múltiples flujos de instrucciones** dentro del mismo código en nuestro programa.

Procesos y núcleos

En nuestros computadores, podemos ver que se ejecutan varios programas o aplicaciones simultáneamente. Esto lo consigue el computador (en verdad, el sistema operativo) a través de la abstracción de un **proceso**. Un proceso es un **programa en ejecución** que ocupa un espacio de la memoria (RAM) del computador, y ejecuta un flujo de instrucciones. Cuando lanzamos un reproductor de música, un navegador web, un editor de texto, nuestra IDE favorita, o nuestro programa de Python, el computador lo ejecuta mediante un **proceso** que sigue un flujo de instrucciones a partir de la primera línea de código.

¿Cómo se ejecutan las instrucciones?, la unidad que ejecuta instrucciones en un computador es la CPU, la cual posee un núcleo (*core*) que lee una instrucción del programa, la ejecuta, y pasa a la siguiente, hasta que las instrucciones se acaban.

Cuando nuestro programa Python está en ejecución (mediante un proceso), hay un núcleo de nuestro computador que está siguiendo ese flujo de instrucciones. Decimos que hay un proceso que ejecuta nuestro programa Python en la CPU.

Los sistemas modernos poseen múltiples núcleos (*cores*). Es común que nuestros computadores, teléfonos celulares y hasta televisores tengan más de un núcleo, y sean ***dualcore***, ***quadcore***, ***octacore***. Eso significa que puede haber varios procesos, uno en cada núcleo y ejecutando de manera realmente simultánea, o paralela. Cada núcleo ejecuta el flujo de instrucciones de un proceso distinto. Cuando hay más procesos que núcleos para ejecutarlos, los procesos se ponen en una cola, esperando su turno para usar un núcleo. El sistema operativo del computador se encarga de que cada núcleo ejecute algunas instrucciones de un proceso, luego lo deje en pausa, y luego ejecute las instrucciones de otros procesos disponibles. Esto ocurre de manera tan rápida, que lo que observamos es que todos los procesos se ejecutan de manera simultánea dentro del mismo núcleo.

Pero cuando tenemos múltiples flujos de instrucciones en un programa, podemos hacer que varios núcleos ejecuten los distintos flujos de instrucciones **de un mismo proceso** al mismo tiempo. Eso es parte del poder que nos entregan los ***threads***.

Paralelismo VS Concurrencia

Antes de profundizar en el concepto de *threads*, es crucial entender la diferencia entre dos conceptos que suelen confundirse: **paralelismo** y **concurrencia**.

El **paralelismo** implica la ejecución simultánea de varias tareas en diferentes núcleos o procesadores. Es decir, operan al mismo tiempo en hardware separado. Es por esto que no es posible obtener un paralelismo real cuando solo se posee un núcleo.

La **concurrencia** consiste en gestionar varias tareas de forma intercalada en un mismo núcleo o procesador. Sin embargo, aunque solo una tarea se ejecute en un instante dado, los cambios de contexto se realizan tan rápido que aparentan simultaneidad.

En Python, la concurrencia se puede alcanzar con *threads* mediante el módulo `threading`. Ahora bien, Python posee un mecanismo llamado *Global Interpreter Lock* (GIL) el cual,

independiente de la cantidad de núcleos que tenga un computador, evita ejecutar *threads* en paralelo. Esto se debe a que un limita la ejecución simultánea de *threads*, ya que se encarga de que solamente un *thread* pueda ejecutar código Python en un instante dado. Para lograr **paralelismo** real en operaciones intensivas de CPU se pueden emplear otras librerías como `multiprocessing`, o, en versiones recientes de python, desactivar manualmente el `GIL`.

Threading

Un ***thread*** (o hilo) es una unidad de ejecución de código dentro de un proceso. Si un proceso es el programa cargado en la memoria del computador, con su código y variables, un ***thread*** lleva la cuenta de en qué parte del código estamos ejecutando.

thread

Todo proceso al crearse posee al menos un ***thread***. A este *thread* le llamamos el *thread principal* del programa. Pero podemos crear más *threads*, y de esa manera hacer que nuestro programa esté ejecutando varias partes del código simultáneamente. Cada ***thread*** lleva su propio registro de variables locales y en qué parte del código del programa se encuentra. Un *thread* también puede crear otros *threads*.

Gracias al sistema operativo, cada *thread* tiene una porción de tiempo para ejecutarse en el núcleo del computador, y podemos ver que varios *threads* se ejecutan de manera simultánea, o *concurrente*. La decisión de qué *thread* se ejecuta a continuación y por cuánto tiempo, se conoce como *thread scheduling* o *time slicing*, que funciona –en general– turnando los *threads* de la siguiente manera:

1. Se escoge un *thread* entre los que están disponible para ejecutar.
2. Se ejecuta ese *thread* durante un cierto tiempo.
3. Se deja el *thread* actual en espera.
4. Se vuelve al paso 1.

Cuando hay un sólo núcleo en el computador, no existe realmente un paralelismo. Sin embargo, los turnos se pasan de manera tan rápida entre los *threads*, que se comporta como si todos estuvieran avanzando en paralelo. Es por esto que dentro de un sólo núcleo, usar *threads* hace que nuestro programa se ejecute de manera más lenta, pues ahora hay que gastar algo de tiempo en controlar los turnos de los *threads*. Sin embargo, la programación es mucho más clara y sencilla cuando usamos *threads* que se hacen cargo de distintos aspectos de nuestro programa, debido a que no debemos programar de forma explícita el salto entre distintas funcionalidades del programa en un solo hilo de ejecución.

Usos de *threads*

Algunos ejemplos donde resulta útil implementar *threads* son:

- **Separar funciones independientes que trabajan sobre un mismo conjunto de datos pero pueden ejecutarse simultáneamente**. Por ejemplo, un *thread* que se encargue de recibir *input* de texto de un usuario y colocarlo en una cola de texto por procesar, y otro *thread* que procese línea a línea y los saque de la cola. Este tipo de patrón, donde existen procesos que se ejecutan secuencialmente con acceso al mismo conjunto de datos pero

que son independientes entre ellos, se conoce como modelo **productor-consumidor** y se ejemplifica en uno de los siguientes *notebooks*.

- **Interfaces gráficas**, donde se necesita interactuar con el usuario mientras se ejecuta algún *thread* de cómputo pesado. Por ejemplo, las interfaces gráficas en un computador: reciben *input* de usuario mediante entradas de texto y botones, mientras otros *threads* actualizan la parte visual simultáneamente.
- **Aplicaciones multiusuario**, en donde cada *thread* se encargaría de las peticiones de cada usuario independientemente. Por ejemplo, un servidor de páginas web debe atender a varios clientes a la vez: cada vez que se conecta un usuario, se debe responder a este de forma correcta, y múltiples usuarios pueden pedir al mismo tiempo una conexión.

Creación de *threads*

En Python 3, el manejo de *threads* se realiza mediante el módulo `threading`, en donde los *threads* son representados por objetos.

```
import threading
```

La clase `Thread` representa un hilo (o *thread*). La intención es que cada hilo ejecute una secuencia de instrucciones específica de manera simultanea al resto de nuestro programa. Para especificar qué instrucciones ejecutará, se le entrega una función, la que se recibe en el parámetro `target` al crear el *thread*. Es decir, cuando el *thread* comience a funcionar, ejecutará la función entregada en este parámetro.

```
def contar_diez_ovejas() -> None:
    print("Tengo sueño...")
    for numero in range(1, 11):
        print(f"({numero} oveja{'s' if numero > 1 else ''})")
    print("A dormir...")

mi_hilo = threading.Thread(target=contar_diez_ovejas)
```

Aquí, se creó una instancia de `Thread`, y se almacenó en la variable `mi_hilo`. Recibió como argumento para ejecutar, la función `contar_diez_ovejas`. Notamos que al crear una instancia de `Thread`, este **NO** se ejecuta automáticamente. Es por eso que no vemos *output* generado, ya que no se están ejecutando las instrucciones de `contar_diez_ovejas`, solo definimos un hilo que puede hacerlo.

Para efectivamente ejecutar el *thread*, se debe llamar al método `start()`. **Es altamente recomendable que éste y todos los ejemplos los revises ejecutando código desde una consola, en lugar de los Jupyter Noteboks, para ver como se comporta.**

```
mi_hilo.start()

Tengo sueño...
(1 oveja)
(2 ovejas)
(3 ovejas)
(4 ovejas)
```

```
(5 ovejas)
(6 ovejas)
(7 ovejas)
(8 ovejas)
(9 ovejas)
(10 ovejas)
A dormir...
```

Es importante notar, que los `Thread` definidos de esta forma son, por defecto, de un solo uso. Es decir, una vez ejecutados, no es posible volver a ejecutarlos. Para eso, es necesario volver a instanciar el *thread*. El siguiente código lanzará un error, ya que el *thread* que intentamos comenzar ya lo hizo antes:

```
mi_hilo.start()
```

```
---------------------------------------------------------------------------
RuntimeError                              Traceback (most recent call last)
Cell In[4], line 1
----> 1 mi_hilo.start()

File /Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/threading.py:989, in Thread.start(self)
    986     raise RuntimeError("thread.__init__() not called")
    988 if self._started.is_set():
--> 989     raise RuntimeError("threads can only be started once")
    991 with _active_limbo_lock:
    992     _limbo[self] = self

RuntimeError: threads can only be started once
```

Como se mencionó inicialmente, todo programa en ejecución tiene al menos un *thread*: el *thread* principal, que corresponde al hilo que ejecuta el flujo principal de nuestro programa. La clase `Thread` admite el argumento `name`, que es un nombre que le podemos dar al *thread* para poder identificarlo. El nombre que se le asigna por defecto al *thread* principal es `"MainThread"`. En el siguiente código, la función `saludar` imprime el nombre del *thread* que lo ejecuta.

```
def saludar() -> None:
    thread_actual = threading.current_thread()
    print(f"Hola desde {thread_actual.name}\n")


hilo_1 = threading.Thread(name="Mi thread 1", target=saludar)
hilo_2 = threading.Thread(name="Mi thread 2", target=saludar)
```

`threading.current_thread()` es una función que retorna **una referencia de la instancia del *thread* que está ejecutando este código** (el *thread* que está en la CPU). Gracias a eso, se puede acceder a atributos de la instancia de `Thread`, como el nombre (`name`). A continuación, veremos que cada *thread* imprime su nombre correspondiente, incluso el *thread* principal:

```
# Llamaremos a saludar() desde los threads nombrados
hilo_1.start()
hilo_2.start()
# Ahora llamaremos a saludar() desde el thread principal
saludar()

Hola desde Mi thread 1

Hola desde Mi thread 2

Hola desde MainThread
```

En el siguiente ejemplo, crearemos tres *threads*: `hilo_lento`, `hilo_rapido_1` e `hilo_rapido_2`, que ejecutarán distintas instancias de las funciones `trabajador_rapido` y `trabajador_lento`.

```
import threading
import time


# Función rápida, que toma 2 segundos
def trabajador_rapido() -> None:
    thread_actual = threading.current_thread()
    print(f"{thread_actual.name} partiendo...")

    # Pondremos a dormir el thread por 2 segundos simulando
    # que ocurre algun proceso dentro de la función
    time.sleep(2)
    print(f"{thread_actual.name} terminando...")

# Función lenta, que toma 6 segundos
def trabajador_lento() -> None:
    thread_actual = threading.current_thread()
    print(f"{thread_actual.name} partiendo...")

    # Ponemos a dormir el thread por 6 segundos simulando
    # un proceso más largo que el anterior dentro de la función
    time.sleep(6)
    print(f"{thread_actual.name} terminando...")


# Creamos los threads usando la clase Thread
hilo_lento = threading.Thread(name="Hilo lento (6s)",
target=trabajador_lento)
```

```
hilo_rapido_1 = threading.Thread(name="Hilo rápido (2s)",
target=trabajador_rapido)
# Usa el nombre asignado por defecto
hilo_rapido_2 = threading.Thread(target=trabajador_rapido)
print("Thread principal: Fueron creados 3 threads")

Thread principal: Fueron creados 3 threads

# Se inicializan los threads creados
print("Thread principal: Empezaré a iniciar los 3 threads")
hilo_rapido_1.start()  # Dormirá por 2 segundos
hilo_rapido_2.start()  # Dormirá por 2 segundos
hilo_lento.start()  # Dormirá por 6 segundos
print("Thread principal: Fueron iniciados 3 threads")
# Todas estas líneas que siguen serán ejecutadas mientras los threads
# se ejecutan independientemente del programa principal

print()
# El thread principal ejecutará lo que queda de código
# mientras los otros 3 threads hacen lo suyo

for i in range(10):
    print(f"Thread principal: Segundo actual: {i}")
    time.sleep(1)

Thread principal: Empezaré a iniciar los 3 threads
Hilo rápido (2s) partiendo...
Thread-6 (trabajador_rapido) partiendo...
Hilo lento (6s) partiendo...
Thread principal: Fueron iniciados 3 threads

Thread principal: Segundo actual: 0
Thread principal: Segundo actual: 1
Hilo rápido (2s) terminando...Thread-6 (trabajador_rapido)
terminando...

Thread principal: Segundo actual: 2
Thread principal: Segundo actual: 3
Thread principal: Segundo actual: 4
Thread principal: Segundo actual: 5
Hilo lento (6s) terminando...
Thread principal: Segundo actual: 6
Thread principal: Segundo actual: 7
Thread principal: Segundo actual: 8
Thread principal: Segundo actual: 9
```

¿Qué pasa con los `print`?

A partir de este ejemplo en adelante, es probable que al ejecutar `print` falten o sobran saltos de línea. La pregunta es, ¿tienen que ver los *threads* con este extraño comportamiento? La respuesta es sí.

De acuerdo a este post en StackOverflow esto se produce porque – a nivel de máquina – escribir el texto del `print` y escribir el salto de línea son **dos instrucciones distintas**. Por lo tanto, es posible que un *thread* imprima el texto, se pause ese *thread*, luego otro *thread* imprima su texto y su respectivo salto de línea, y luego al volver al primer *thread* este imprima el salto de línea que faltaba.

En el ejemplo, vemos que una vez inicializados los *threads*, **el programa principal continúa con el resto de instrucciones mientras los *threads* se ejecutan**. Luego los tres *threads* terminan independientemente en distintos momentos. El programa principal esperará a que todos los *threads* hayan terminado correctamente.

También es posible, mediante el atributo `args` o `kwargs`, pasar los argumentos a la función definida como `target` del *thread*.

```
import threading
import time


def contar_ovejas_hasta(max_ovejas: int) -> None:
    thread_actual = threading.current_thread()
    print(f"{thread_actual.name} tiene sueño...")
    for numero in range(1, max_ovejas + 1):
        time.sleep(1)
        print(f"({thread_actual.name}: {numero} oveja{'s' if numero >
1 else ''})")
    print(f"{thread_actual.name} a dormir...")


# Se crean los threads usando la clase Thread, asociada a la función
objetivo para
# ser ejecutada por el thread, y los atributos de la función son
ingresados en
# args o kwargs

t1 = threading.Thread(name="Thread 1", target=contar_ovejas_hasta,
args=(10,))
t2 = threading.Thread(name="Thread 2", target=contar_ovejas_hasta,
kwargs={"max_ovejas": 15})
t1.start()
t2.start()
```

```
Thread 1 tiene sueño...Thread 2 tiene sueño...

(Thread 1: 1 oveja)
(Thread 2: 1 oveja)
(Thread 1: 2 ovejas)
(Thread 2: 2 ovejas)
(Thread 1: 3 ovejas)(Thread 2: 3 ovejas)

(Thread 2: 4 ovejas)(Thread 1: 4 ovejas)
```

```
(Thread 1: 5 ovejas)(Thread 2: 5 ovejas)

(Thread 1: 6 ovejas)
(Thread 2: 6 ovejas)
(Thread 1: 7 ovejas)(Thread 2: 7 ovejas)

(Thread 2: 8 ovejas)(Thread 1: 8 ovejas)

(Thread 1: 9 ovejas)(Thread 2: 9 ovejas)

(Thread 2: 10 ovejas)
(Thread 1: 10 ovejas)
Thread 1 a dormir...
(Thread 2: 11 ovejas)
(Thread 2: 12 ovejas)
(Thread 2: 13 ovejas)
(Thread 2: 14 ovejas)
(Thread 2: 15 ovejas)
Thread 2 a dormir...
```

En concordancia con el modelamiento basado en OOP, podemos crear también *threads* como instancias de una clase que hereda desde la clase `Thread`. Es decir, podemos crear clases para *threads* con comportamiento común. Para definir las instrucciones que serán ejecutadas por la instancia de *thread*, se debe hacer *override* del método `run()` en la clase, el cual se ejecuta luego de llamar a `start`, al igual que antes. Con esto, ahora podremos personalizar este método para definir las instrucciones a ejecutar directamente para cada clase de *thread*.

```python
import threading
import time


class CuentaOvejas(threading.Thread): # Hereda de Thread
    """Este será nuestro nuevo Cuenta Ovejas basado en Thread"""

    def __init__(self, nombre: str, max_ovejas: int) -> None:
        # En el caso de los threads, lo primero es invocar al init
original. SIEMPRE.
        super().__init__(name=nombre)
        self.max_ovejas = max_ovejas # Se agrega un atributo de
instancia extra

    def run(self) -> None:
        # Este metodo define las instrucciones a ejecutar de este
thread
        # cuando ejecutamos el metodo start()
        print(f"\t{self.name} tiene sueño...")
        tiempo_partida = time.time()
        for numero in range(1, self.max_ovejas + 1):
            time.sleep(1)
```

```
            print(f"\t({self.name}: {numero} oveja{'s' if numero > 1
else ''})")
        print(f"\t{self.name} a dormir...")
        print(f"\t{self.name} se durmió después de {time.time() -
tiempo_partida} seg.")


class CuentaLiebres(threading.Thread): # Hereda de Thread
    """
    Este será un nuevo Cuenta Liebres basado en Thread
    Las liebres son más rápidas, así que cuenta dos por segundo
    """

    def __init__(self, nombre: str, max_liebres: int) -> None:
        super().__init__(name=nombre)
        self.max_liebres = max_liebres

    def run(self) -> None:
        print(f"\t{self.name} tiene sueño...")
        tiempo_partida = time.time()
        for numero in range(1, self.max_liebres + 1):
            if numero % 2 == 1:
                time.sleep(1)
            print(f"\t({self.name}: {numero} liebre{'s' if numero > 1
else ''})")
        print(f"\t{self.name} a dormir...")
        print(f"\t{self.name} se durmió después de {time.time() -
tiempo_partida} seg.")


# Se crean los threads
cuenta_ovejas = CuentaOvejas("Gogeta", 10)
cuenta_liebres = CuentaLiebres("Anya", 10)
print("Thread principal: Fueron creados 2 threads")
```

```
Thread principal: Fueron creados 2 threads
```

```
# Se inicializan los threads creados
cuenta_ovejas.start()
cuenta_liebres.start()
print("Thread principal: Fueron iniciados 2 threads")
# Todas estas líneas serán ejecutadas mientras los threads se ejecutan
# independientemente del programa principal

print()
# El thread principal ejecutará lo que queda código
# mientras los otros 2 threads hacen lo suyo
for i in range(10):
    print(f"Thread principal: Segundo actual: {i}")
    time.sleep(1)
```

```
     Gogeta tiene sueño...
     Anya tiene sueño...
Thread principal: Fueron iniciados 2 threads

Thread principal: Segundo actual: 0
     (Gogeta: 1 oveja)     (Anya: 1 liebre)
     (Anya: 2 liebres)

Thread principal: Segundo actual: 1
     (Anya: 3 liebres)Thread principal: Segundo actual: 2

     (Anya: 4 liebres)
     (Gogeta: 2 ovejas)
Thread principal: Segundo actual: 3   (Anya: 5 liebres)
     (Anya: 6 liebres)
     (Gogeta: 3 ovejas)

     (Anya: 7 liebres)     (Gogeta: 4 ovejas)

     (Anya: 8 liebres)
Thread principal: Segundo actual: 4
     (Gogeta: 5 ovejas)    (Anya: 9 liebres)
     (Anya: 10 liebres)
     Anya a dormir...
     Anya se durmió después de 5.021545886993408 seg.
Thread principal: Segundo actual: 5

Thread principal: Segundo actual: 6   (Gogeta: 6 ovejas)

Thread principal: Segundo actual: 7   (Gogeta: 7 ovejas)

     (Gogeta: 8 ovejas)Thread principal: Segundo actual: 8

     (Gogeta: 9 ovejas)
Thread principal: Segundo actual: 9
     (Gogeta: 10 ovejas)
     Gogeta a dormir...
     Gogeta se durmió después de 10.042298793792725 seg.
```

Esperando otros *threads*: `join()`

Hasta ahora hemos visto comportamiento de *threads* que se ejecutan "en paralelo" y de forma independiente al flujo del programa principal. Pero hay ocasiones donde el programa principal puede necesitar que otros *threads* terminen antes de ejecutar ciertas instrucciones.

Por ejemplo, un programa que debe obtener el total de compras de una tienda, donde el detalle de las compras se encuentran en cinco archivos en distintos formatos. El programa delega a cinco *threads* distintos el leer y sumar el total de compras de cada archivo, **espera** a que cada *thread* termine de hacer sus cálculos y luego obtiene los cinco resultados y los suma.

Si necesitamos que el programa principal espere a que termine la ejecución de algún *thread* (o grupo de *threads*), debemos usar el método `join(timeout=None)` después de que el *thread* ha sido iniciado mediante `start()`.

Al usar `join()`, **el *thread* que llama al método** queda bloqueado hasta que los *threads* referenciados terminen correctamente. No obstante, si se especifica un `timeout` (en segundos) el programa esperará al *thread* referenciado solo por ese tiempo y luego continuará.

La figura siguiente explica gráficamente la ejecución de un programa usando `join()`.

join

En el código esto se refleja como que *Main Thread* incluye las llamadas `T1.join()` y `T2.join()`. Como es *Main Thread* quien llama a `join`, es ese el *thread* que se bloquea esperando a otro; primero espera a `T1` y luego a `T2`. Por otro lado, no sólo el *Main Thread* puede esperar: **cualquier *thread* puede esperar a cualquier otro** usando `join()`.

Veamos ahora una extensión del ejemplo anterior pero con la incorporación de `join()`. Aquí, los ayudantes del curso esperan a que los profesores (o casi todos) se duerman para empezar a festejar. Notar que el comportamiento de los ayudantes está en el programa principal, y el festejo no comienza hasta después de que la mayoría de los *threads* ejecutados terminan.

```
import threading
import time


# Usamos la definicion de los Thread declarados en el ejemplo anterior
# Se crean los threads usando la clase Thread.
naruto = CuentaOvejas("Naruto", 5)
goku = CuentaOvejas("Goku", 2)
komi = CuentaLiebres("Komi", 5)
anya = CuentaLiebres("Anya", 7)
chisato = CuentaLiebres("Chisato", 20)

# Se inicializan los threads creados
naruto.start()
goku.start()
komi.start()
anya.start()
chisato.start()

print("\nMinions: Los protagonistas se fueron a dormir...")

# Aquí incorporamos el método join() para bloquear el programa
principal
goku.join()  # No especificamos timeout, esperará lo que sea necesario
print("Minions: ¡GOKU SE DURMIÓ!")

naruto.join() # No especificamos timeout, esperará lo que sea
necesario
print("Minions: ¡NARUTO SE DURMIÓ!")
```

```
komi.join() # No especificamos timeout, esperará lo que sea necesario
print("Minions: ¡KOMI SE DURMIÓ!")

anya.join() # No especificamos timeout, esperará lo que sea necesario
print("Minions: ¡ANYA SE DURMIÓ!")

chisato.join(1)  # Esperaremos máximo 1 segundo después del último
dormido, ya es muy tarde
print("Minions: ¡(casi todos) los protagonistas se durmieron! ¡A
festejar!")

# En este punto, el programa ha esperado por los cuatro threads que
creamos
# Estas líneas serán ejecutadas después de que los threads hayan
terminado
for _ in range(6):
    print("Minions: ♪♫♪♫♪♫♪♫♪♫♪♫♪")
    time.sleep(1)

print("\nMinions: Ojalá no nos hayan escuchado...")
```

```
     Naruto tiene sueño...The history saving thread hit an unexpected
error (OperationalError('attempt to write a readonly
database')).History will not be written to the database.
     Goku tiene sueño...

     Komi tiene sueño...
     Anya tiene sueño...
     Chisato tiene sueño...

Minions: Los protagonistas se fueron a dormir...
     (Goku: 1 oveja) (Naruto: 1 oveja)
     (Komi: 1 liebre)
     (Komi: 2 liebres)
     (Anya: 1 liebre)
     (Anya: 2 liebres)
     (Chisato: 1 liebre)
     (Chisato: 2 liebres)

     (Naruto: 2 ovejas)    (Anya: 3 liebres)
     (Anya: 4 liebres)

     (Komi: 3 liebres)
     (Komi: 4 liebres)
     (Chisato: 3 liebres)
     (Chisato: 4 liebres)
     (Goku: 2 ovejas)
     Goku a dormir...
     Goku se durmió después de 2.014195203781128 seg.
```

```
Minions: ¡GOKU SE DURMIÓ!
     (Naruto: 3 ovejas)    (Komi: 5 liebres)
     Komi a dormir...
     Komi se durmió después de 3.010962963104248 seg.
     (Chisato: 5 liebres)
     (Chisato: 6 liebres)

     (Anya: 5 liebres)
     (Anya: 6 liebres)
     (Chisato: 7 liebres)
     (Chisato: 8 liebres)
     (Naruto: 4 ovejas)
     (Anya: 7 liebres)
     Anya a dormir...
     Anya se durmió después de 4.016746997833252 seg.
     (Naruto: 5 ovejas)    (Chisato: 9 liebres)
     (Chisato: 10 liebres)

     Naruto a dormir...
     Naruto se durmió después de 5.018253803253174 seg.
Minions: ¡NARUTO SE DURMIÓ!
Minions: ¡KOMI SE DURMIÓ!
Minions: ¡ANYA SE DURMIÓ!
     (Chisato: 11 liebres)
     (Chisato: 12 liebres)
Minions: ¡(casi todos) los protagonistas se durmieron! ¡A festejar!
Minions: ♪♫♪♫♪♫♪♫♪♫♪♫♪♫♪
     (Chisato: 13 liebres)Minions: ♪♫♪♫♪♫♪♫♪♫♪♫♪

     (Chisato: 14 liebres)
Minions: ♪♫♪♫♪♫♪♫♪♫♪♫♪♫♪  (Chisato: 15 liebres)
     (Chisato: 16 liebres)

     (Chisato: 17 liebres)Minions: ♪♫♪♫♪♫♪♫♪♫♪♫♪

     (Chisato: 18 liebres)
Minions: ♪♫♪♫♪♫♪♫♪♫♪♫♪♫♪
     (Chisato: 19 liebres)
     (Chisato: 20 liebres)
     Chisato a dormir...
     Chisato se durmió después de 10.03629994392395 seg.
Minions: ♪♫♪♫♪♫♪♫♪♫♪♫♪♫♪

Minions: Ojalá no nos hayan escuchado...
```

¿Qué ocurre si tenemos lo siguiente?

```
thread_1.start()
thread_1.join()
thread_2.start()
```

```
thread_2.join()
thread_3.start()
thread_3.join()
```

¿Tendremos multiples *threads* ejecutandose?

La respuesta es **no**. En este código, se está empezando un *thread* y se esta esperando que finalice (`.join()`) antes de continuar con la siguiente linea de código que se encarga de empezar el siguiente *thread*. Cuando trabajamos con *threads* hay que tener mucho cuidado en la ubicación de los `.start()` y los `.join()`. El comportamiento estándar sería empezar todos los *threads* y luego comenzar con los `join()` para esperarlos.

`is_alive()`

Es posible identificar si un *thread* todavía está en funcionamiento mediante el uso del método `is_alive()`. Por lo general este método se implementa para saber el estado del *thread* después del uso de `join()`, al cual se le ha definido su tiempo máximo de espera.

```
import threading
import time


# Usamos la definicion de los Thread declarados en el ejemplo anterior
# Se crean los threads usando la clase Thread.
naruto = CuentaOvejas("Naruto", 5)
goku = CuentaOvejas("Goku", 2)
komi = CuentaLiebres("Komi", 5)
anya = CuentaLiebres("Anya", 7)
chisato = CuentaLiebres("Chisato", 20)

# Se inicializan los threads creados
naruto.start()
goku.start()
komi.start()
anya.start()
chisato.start()

print("\nMinions: Los protagonistas se fueron a dormir...")

# Aquí incorporamos el método join() para bloquear el programa
principal
goku.join()  # No especificamos timeout, esperará lo que sea necesario
print("Minions: ¡GOKU SE DURMIÓ!")

naruto.join() # No especificamos timeout, esperará lo que sea
necesario
print("Minions: ¡NARUTO SE DURMIÓ!")

komi.join() # No especificamos timeout, esperará lo que sea necesario
print("Minions: ¡KOMI SE DURMIÓ!")
```

```
anya.join() # No especificamos timeout, esperará lo que sea necesario
print("Minions: ¡ANYA SE DURMIÓ!")

chisato.join(1)  # Esperaremos máximo 1 segundo después del último
dormido, ya es muy tarde

if chisato.is_alive():
    print("Minions: Chisato sigue despierta ☹. A la casa amigos!!!")
else:
    print("Minions: ¡Todos los protagonistas se durmieron! ¡A
festejar!")
    for i in range(6):
        print("Ayudantes: ♪♫♪♫♪♫♪♫♪♫♪♫♪♫")
        time.sleep(1)
```

```
     Naruto tiene sueño...
     Goku tiene sueño...
     Komi tiene sueño...
     Anya tiene sueño...
     Chisato tiene sueño...

Minions: Los protagonistas se fueron a dormir...
     (Naruto: 1 oveja)
     (Goku: 1 oveja)
     (Komi: 1 liebre)
     (Komi: 2 liebres)
     (Anya: 1 liebre)
     (Anya: 2 liebres)
     (Chisato: 1 liebre)
     (Chisato: 2 liebres)
     (Naruto: 2 ovejas)    (Komi: 3 liebres)
     (Komi: 4 liebres)

     (Goku: 2 ovejas)
     Goku a dormir...
     Goku se durmió después de 2.009692907333374 seg.
     (Anya: 3 liebres)
     (Anya: 4 liebres)
     (Chisato: 3 liebres)
     (Chisato: 4 liebres)
Minions: ¡GOKU SE DURMIÓ!
     (Komi: 5 liebres)     (Naruto: 3 ovejas)
     (Anya: 5 liebres)
     (Anya: 6 liebres)
     (Chisato: 5 liebres)
     (Chisato: 6 liebres)

     Komi a dormir...
     Komi se durmió después de 3.0137557983398438 seg.
```

```
      (Anya: 7 liebres)     (Naruto: 4 ovejas)
      (Chisato: 7 liebres)
      (Chisato: 8 liebres)

      Anya a dormir...
      Anya se durmió después de 4.019599914550781 seg.
      (Chisato: 9 liebres)
      (Chisato: 10 liebres)
      (Naruto: 5 ovejas)
      Naruto a dormir...
      Naruto se durmió después de 5.027347087860107 seg.
Minions: ¡NARUTO SE DURMIÓ!
Minions: ¡KOMI SE DURMIÓ!
Minions: ¡ANYA SE DURMIÓ!
      (Chisato: 11 liebres)
      (Chisato: 12 liebres)
Minions: Chisato sigue despierta ☹. A la casa amigos!!!
      (Chisato: 13 liebres)
      (Chisato: 14 liebres)
      (Chisato: 15 liebres)
      (Chisato: 16 liebres)
      (Chisato: 17 liebres)
      (Chisato: 18 liebres)
      (Chisato: 19 liebres)
      (Chisato: 20 liebres)
      Chisato a dormir...
      Chisato se durmió después de 10.046483993530273 seg.
```

```
print("(Minions vuelven)")
print("Minions: Esperemos un poco más...")
time.sleep(2)
for protagonista in [goku, chisato, komi, naruto, anya]:
    if protagonista.is_alive():
        print(f"Minions: {protagonista.name} aún está despiert@ ☹")
    else:
        print(f"Minions: ¡{protagonista.name} se quedó dormid@!")
```

```
(Minions vuelven)
Minions: Esperemos un poco más...
Minions: ¡Goku se quedó dormid@!
Minions: ¡Chisato se quedó dormid@!
Minions: ¡Komi se quedó dormid@!
Minions: ¡Naruto se quedó dormid@!
Minions: ¡Anya se quedó dormid@!
```

Daemons

En los ejemplos anteriores, el programa espera a que todos los *threads* terminen su trabajo antes de terminar su ejecución. Los ***daemon threads*** son aquellos que, **a pesar de que estén aún corriendo, no impiden que el programa principal termine**.

Si no existieran los *daemon threads*, tendríamos que preocuparnos de tener en cuenta todos nuestros *threads* y preocuparnos de terminarlos a todos para que el programa principal pueda terminar. Con los *daemon threads*, podemos ponerlos a correr y olvidarnos de ellos, y cuando el programa termina, los *daemon threads* serán terminados inmediatamente. El programa principal terminará cuando todos los *threads no-daemon* hayan finalizado.

Para especificar que un *thread* debe ser *daemon*, se especifica el parámetro `daemon=True` en el constructor del *thread*. Una forma alternativa es escribir el atributo `daemon` del *thread*, después de haberlo creado.

Warning: este notebook no refleja el efecto de *daemon*.

Debido al funcionamiento interno de los Jupyter Notebooks, no se puede notar el efecto de utilizar esta propiedad de los *threads* dentro en las celdas del *notebook*. Es por esto que el siguiente código se encuentra en el archivo adjunto `daemon_threads.py`, ubicado en la carpeta `scripts`, para ser ejecutado **fuera de este *notebook*, desde una consola**.

```
import threading
import time


def dormilon() -> None:
    print(f"{threading.current_thread().name} tiene sueño...")
    time.sleep(2)
    print(f"{threading.current_thread().name} se durmió.")


def con_insomnio() -> None:
    print(f"{threading.current_thread().name} tiene sueño...")
    time.sleep(10)
    print(f"{threading.current_thread().name} se durmió.")


# Forma 1 de hacer un thread daemon
dormilon = threading.Thread(name="Dormilón", target=dormilon,
daemon=True)
# Forma 2 de hacer un thread daemon
con_insomnio = threading.Thread(name="Con insomnio",
target=con_insomnio)
con_insomnio.daemon = True

# Se inicializan los threads
dormilon.start()
con_insomnio.start()
```

La siguiente figura muestra la salida en la consola para el programa **con** los *threads* configurados como *daemon*. Los *threads* se interrumpen abruptamente, luego de que el programa principal termina su ejecución.

daemon

Comparemos el resultado anterior con la salida en la consola para el programa **sin** los *threads* configurados como *daemon* (`daemon=False`), como muestra la imagen siguiente. Podemos ver que la ejecución de los *threads* fue completada y que el programa no se cierra hasta que los *threads* terminan.

non_daemon

En la penúltima imagen, podemos apreciar que el programa principal termina antes que los *daemon threads*. Es posible hacer que el programa espere a un *daemon thread* mediante el uso de `join()`, lo que tendría sentido cuando debemos esperar algún resultado del *thread* para el resto del programa.

```
import threading
import time


def dormilon() -> None:
    print(f"{threading.current_thread().name} tiene sueño...")
    time.sleep(2)
    print(f"{threading.current_thread().name} se durmió.")


def con_insomnio() -> None:
    print(f"{threading.current_thread().name} tiene sueño...")
    time.sleep(10)
    print(f"{threading.current_thread().name} se durmió.")


# Forma 1 de hacer un thread daemon
dormilon = threading.Thread(name="Dormilón", target=dormilon,
daemon=True)
# Forma 2 de hacer un thread daemon
con_insomnio = threading.Thread(name="Con insomnio",
target=con_insomnio)
con_insomnio.daemon = True

# Se inicializan los threads
dormilon.start()
con_insomnio.start()

# Esperamos los threads.
# Lo esperamos por una cantidad indefinida de tiempo
dormilon.join()
# Esperamos sólo 5 segundos
con_insomnio.join(5)
```

```
Dormilón tiene sueño...
Con insomnio tiene sueño...
Dormilón se durmió.
Con insomnio se durmió.
[02:58:16] Hola :) acabas de ejecutar el timer 2
```

```
[02:58:19] ¡Hola!, acabas de ejecutar el timer 1
[02:58:45] Hola :) acabas de ejecutar el timer 2
[02:58:48] ¡Hola!, acabas de ejecutar el timer 1
```

Un aspecto importante a notar es que una vez que se ha llamado el método `start()` un *thread* no puede cambiar desde *daemon* a *no-daemon*, o vicecersa. Si tratamos de hacerlo, se lanza una excepción del tipo `RuntimeError`.

```
dormilon = threading.Thread(name="Dormilón", target=dormilon,
daemon=True)
dormilon.start()

# Noo!
dormilon.daemon = False

dormilon.join()
```

```
Exception in thread Dormilón:
Traceback (most recent call last):
  File
"/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
threading.py", line 1073, in _bootstrap_inner
    self.run()
  File
"/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
site-packages/ipykernel/ipkernel.py", line 766, in run_closure
    _threading_Thread_run(self)
  File
"/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
threading.py", line 1010, in run
    self._target(*self._args, **self._kwargs)
TypeError: 'Thread' object is not callable

---------------------------------------------------------------------
-----
RuntimeError                              Traceback (most recent call
last)
Cell In[7], line 5
      2 dormilon.start()
      4 # Noo!
----> 5 dormilon.daemon = False
      7 dormilon.join()

File
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
threading.py:1256, in Thread.daemon(self, daemonic)
   1254     raise RuntimeError('daemon threads are disabled in this
interpreter')
   1255 if self._started.is_set():
-> 1256     raise RuntimeError("cannot set daemon status of active
```

```
thread")
   1257 self._daemonic = daemonic

RuntimeError: cannot set daemon status of active thread
```

Al hacer una subclase de `Thread`, que queremos que sea *daemon*, podemos indicar que el *thread* es *daemon* durante su inicialización, usando el atributo `daemon`.

```
class Daemon(threading.Thread):

    def __init__(self) -> None:
        super().__init__()
        # Cuando inicializamos el thread lo declaramos como daemon
        self.daemon = True

    def run(self):
        print("Daemon thread: Empezando...")
        time.sleep(2)
        print("Daemon thread: Terminando...")


daemon = Daemon()
daemon.start()
daemon.join()

Daemon thread: Empezando...
Daemon thread: Terminando...
```

Ahora que ya entendemos que un thread daemon se puede interrumpir su ejecución si el programa principal se acaba. Vamos a revisar el siguiente código que está en `daemon_threads_print_error`, dentro de la carpeta `scripts`.

```
from threading import Thread
import time

def presentar(i):
    print("Hola soy el thread {}".format(i, i))

def main():
    # Crear threads
    threads = []
    for i in range(25):
        threads.append(Thread(target=presentar, daemon=True,
args=(i+1,)))

    # Empezar los threads
    for thread in threads:
        thread.start()
```

```
if __name__ == "__main__":
    print("Empezar programa, crearé 25 threads")
    main()
```

En este código, se crean 25 *thread daemon* los cuales se van a presentar y se acaba todo. Si ejecutamos este archivo en consola, un posible resultado es:

daemon_error

La última parte da miedo... ¿que es eso de `Fatal Python error`? ¿por qué no vemos el `print` del **thread 23**?

Este error se debe a que un *thread* estaba junto en proceso de imprimir el string en consola cuando fue interrumpido por el programa principal. Dado que imprimir en consola implica enviarle al sistema operativo una acción, se genera este error porque fue gatillada una interrumpición al sistema operativo por parte del hilo principapl. Este error es normal que ocurra si es que estamos dejando que algún *daemon thread* maneje `prints` y luego cortamos el programa principal de forma abrupta.

Timers

La clase `Timer` es una subclase de la clase `Thread` y permite **ejecutar un proceso o acción después que ha pasado un determinado tiempo**. Es decir, al ser ejecutado, espera el tiempo que se le especifica y luego ejecuta las instrucciones determinadas. `Timer` requiere, como parámetros básicos, el tiempo en segundos después del cual debe ejecutarse el *thread*, la función a ejecutar, y los argumentos de la función a ejecutar.

Warning: este notebook podría no reflejar el efecto de un *Timer*. Recomendamos crear un archivo .py con el código a continuación y ejecutarlo para ver el efecto esperado.

Según la versión de jupyter notebook que dispongan, puede que no se muestre el efecto de utilizar esta propiedad de los *threads* en las celdas del notebook, es decir, que los *threads* no esperen el tiempo necesario antes de ejecutarse. En caso que ocurra eso, recomendamos probar el código de la siguiente celda en una consola y ejecutar desde ahí.

```
from os.path import join
import time

def tiempo_actual() -> str:
    return time.strftime("%H:%M:%S")


def mi_timer(ruta_archivo: str) -> None:
    with open(ruta_archivo) as archivo:
        for linea in archivo:
            print(f"[{tiempo_actual()}] {linea.strip()}")

t1 = threading.Timer(4.0, mi_timer, args=(join("files",
```

```
"mensaje_01.txt"), ))
t2 = threading.Timer(1.0, mi_timer, kwargs={"ruta_archivo":
join("files", "mensaje_02.txt")})
print("Se han creado 2 Timers")
```

```
Se han creado 2 Timers
```

```
print(f"[{tiempo_actual()}] Activando timer 1 que comenzará luego de 4
segundos")
t1.start() # el thread t1 comenzará después de 4 seconds

print(f"[{tiempo_actual()}] Activando timer 2 que comenzará luego de 1
segundos")
t2.start() # el thread t2 comenzará después de 1 seconds
```

```
[02:58:15] Activando timer 1 que comenzará luego de 4 segundos
[02:58:15] Activando timer 2 que comenzará luego de 1 segundos
```

Una ventaja del Timer es que, una vez hecho el `.start()`, podemos cancelar la ejecución del `Timer` con el método `.cancel()`

```
t1 = threading.Timer(4.0, mi_timer, args=("files/mensaje_01.txt",))

print(f"[{tiempo_actual()}] Activando timer 1 que comenzará luego de 4
segundos")
t1.start() # el thread t1 comenzará después de 4 seconds

time.sleep(3)

print(f"[{tiempo_actual()}] No vamos ver nada del contenido del timer
1 ♂")
t1.cancel()
```

```
[02:58:23] Activando timer 1 que comenzará luego de 4 segundos
[02:58:26] No vamos ver nada del contenido del timer 1 ♂
```

Cómo se mencionó antes, un `Timer` es solo un thread que espera una cantidad de tiempo antes de empezar. Por lo tanto, podemos simular este comportamiento con la misma clase `Thread`.

```
import time

def tiempo_actual() -> str:
    return time.strftime("%H:%M:%S")

class MiTimer(threading.Thread):

    def __init__(self, tiempo_espera: int, ruta_archivo: str) -> None:
        super().__init__()
        self.tiempo_espera = tiempo_espera
```

```
        self.ruta_archivo = ruta_archivo
        self._cancel = False

    def cancel(self) -> None:
        self._cancel = True

    def run(self) -> None:
        time.sleep(self.tiempo_espera)
        if self._cancel == True:
            return

        with open(self.ruta_archivo) as archivo:
            for linea in archivo:
                print(f"[{tiempo_actual()}] {linea.strip()}")


t1 = MiTimer(4, "files/mensaje_01.txt")
t2 = MiTimer(1, ruta_archivo = "files/mensaje_02.txt")

print(f"[{tiempo_actual()}] Activando timer 1 que comenzará luego de 4
segundos")
t1.start() # el thread t1 comenzará después de 4 seconds

print(f"[{tiempo_actual()}] Activando timer 2 que comenzará luego de 1
segundos")
t2.start() # el thread t2 comenzará después de 1 seconds
```

```
[02:58:44] Activando timer 1 que comenzará luego de 4 segundos
[02:58:44] Activando timer 2 que comenzará luego de 1 segundos
```

```
t1 = MiTimer(4, "files/mensaje_01.txt")
t2 = MiTimer(1, ruta_archivo = "files/mensaje_02.txt")

print(f"[{tiempo_actual()}] Activando timer 1 que comenzará luego de 4
segundos")
t1.start() # el thread t1 comenzará después de 4 seconds

print(f"[{tiempo_actual()}] Activando timer 2 que comenzará luego de 1
segundos")
t2.start() # el thread t2 comenzará después de 1 seconds

print(f"[{tiempo_actual()}] Vamos a cancelar la ejecución de los 2
timers")
t1.cancel()
t2.cancel()
```

```
[02:58:50] Activando timer 1 que comenzará luego de 4 segundos
[02:58:50] Activando timer 2 que comenzará luego de 1 segundos
[02:58:50] Vamos a cancelar la ejecución de los 2 timers
```